



A Denoising Diffusion-Based Evolutionary Algorithm Framework: Application to the Maximum Independent Set Problem

Joan Salvà Soler^(✉)  and Günther R. Raidl 

Algorithms and Complexity Group, TU Wien, Vienna, Austria
joan.salvasoler@h2o.ai, raidl@ac.tuwien.ac.at

Abstract. Denoising diffusion models (DDMs) offer a promising generative approach for combinatorial optimization, yet they often lack the robust exploration capabilities of traditional metaheuristics like evolutionary algorithms (EAs). We propose a Denoising Diffusion-based Evolutionary Algorithm (DDEA) framework that synergistically integrates these paradigms. It utilizes pre-trained DDMs for both high-quality and diverse population initialization and a novel diffusion-based recombination operator, trained via imitation learning against an optimal demonstrator. Evaluating DDEA on the Maximum Independent Set problem on Erdős-Rényi graphs, we demonstrate notable improvements over DIFUSCO, a leading DDM solver. DDEA consistently outperforms it given the same time budget, and surpasses Gurobi on larger graphs under the same time limit, with DDEA's solution sizes being 3.9% and 7.5% larger on the ER-300-400 and ER-700-800 datasets, respectively. In out-of-distribution experiments, DDEA provides solutions of 11.6% higher quality than DIFUSCO under the same time limit. Ablation studies confirm that both diffusion initialization and recombination are crucial. Our work highlights the potential of hybridizing DDMs and EAs, offering a promising direction for the development of powerful machine learning solvers for complex combinatorial optimization problems.

Keywords: Denoising Diffusion Models · Evolutionary Algorithms · Combinatorial Optimization · Maximum Independent Set Problem

1 Introduction

Combinatorial Optimization (CO) problems involve finding an optimal solution from a discrete but possibly huge set of candidates. Many CO problems are NP-hard and challenging to solve in practice for practically relevant instances. Traditional approaches to tackle such problems often rely on methods like Integer Linear Programming (ILP) or carefully designed (meta-)heuristics, which often require substantial domain expertise and may struggle with scalability.

Advances in deep learning have led to new methods for tackling CO problems [4], with Graph Neural Networks (GNNs) emerging as a foundational architecture for many of these approaches, owing to the inherently graph-structured nature of many CO problems [11]. More recently, generative models like Denoising Diffusion Models (DDMs) have demonstrated competitive performance on CO problems, reaching state-of-the-art performance among neural solvers for the Traveling Salesman Problem (TSP) [18, 24, 28]. Frameworks such as DIFUSCO [24] aim to efficiently generate diverse, high-quality candidate solutions in parallel by leveraging DDMs trained in a supervised manner from solved instances. However, these diffusion-based methods still face significant challenges, including ensuring the feasibility of generated solutions and generalizing effectively to different problem sizes or variants. These methods are often reliant on problem-specific post-processing heuristics to achieve competitive performance.

This reliance on subsequent, often problem-specific, refinement stages highlights a critical gap. While DDMs excel at efficiently generating potentially diverse solutions, they frequently lack the robust, structured exploration mechanisms inherent in classical metaheuristics. Conversely, metaheuristics like evolutionary algorithms (EAs) provide a general framework for robust search but can struggle without effective methods to initialize populations with high-quality, diverse individuals or to generate promising offspring through recombination and mutation. Integrating the generative strengths of DDMs with the structured search of EAs presents an opportunity to overcome the individual limitations.

To bridge this gap, we propose the Denoising Diffusion-Based Evolutionary Algorithm (DDEA), a novel hybrid framework that integrates pre-trained DDMs into core operators of an EA. Specifically, we (1) use the DDM to initialize a diverse population of high-quality solutions for exploration and (2) train a novel diffusion-based recombination operator using imitation learning from an ILP-based demonstrator for exploitation. This synergy aims to combine the efficient generation of diverse, high-quality solutions from DDMs with the problem-agnostic exploratory capabilities of EAs. Our DDEA goes beyond using DDMs merely as a solution generator followed by refinement. Instead, the refinement is embedded within the evolutionary loop, using DDM inference to improve the quality of the offspring. We demonstrate and experimentally evaluate the effectiveness of the DDEA on the Maximum Independent Set Problem (MISP).

The remainder of this paper is structured as follows: Sect. 2 discusses related work. Section 3 details the proposed DDEA framework. Section 4 presents the experimental setup and results. Finally, Sect. 5 concludes the paper and discusses future work.

2 Related Work

This section reviews literature relevant to machine learning for combinatorial optimization, with a focus on denoising diffusion-based methods and DIFUSCO. We also briefly review evolutionary algorithms and hybrid ML-EA approaches.

Machine Learning approaches for CO can be broadly categorized along two axes: the type of learning employed and the nature of algorithmic integration with CO frameworks [4].

From the learning type perspective, imitation learning trains models to mimic expert functions, often used to approximate expensive components like MIP branching decisions [15]. Reinforcement learning (RL) enables agents to learn policies through trial-and-error interaction with the optimization environment, successfully applied to problems like the traveling salesman problem (TSP) [16]. Gradient-based learning uses neural networks to parameterize the solution space and optimizes directly using continuous relaxations, representing an unsupervised approach [12, 22].

From the algorithmic integration perspective, end-to-end learning trains a single ML model to map problem instances directly to solutions, exemplified by pointer networks for TSP [26], autoregressive approaches like transformer-based models [16] and generative flow network models [29], and denoising diffusion-based models like DIFUSCO [24]. Another paradigm is learning to configure algorithms, which uses ML to tune parameters or select heuristics for existing CO solvers, such as predicting good MIP solver settings [13] or using LLMs to propose heuristic components [27]. Finally, ML alongside optimization algorithms involves tight, iterative coupling where the ML model guides the CO solver’s decisions, like predicting branching variables or node selection strategies in branch-and-bound [23].

Across many of these ML approaches, particularly those tackling problems with inherent graph structures, graph neural networks (GNNs) have become a foundational architectural component [8, 11]. Their capacity to process graph data efficiently allows them to learn representations that effectively capture the combinatorial relationships within CO problem instances, making them suitable for diverse tasks from RL policy learning [14] to end-to-end solution generation [12, 24]. Many CO problems have an inherent graph structure, which makes GNNs a natural fit for these problems. The work by [8] surveys the use of GNNs in CO, while [11] proposes a unified GNN framework for CO problems that includes techniques for graph representation, converting non-graph problems to graph form, graph decomposition, and graph simplification to enable efficient GNN-based solutions.

Denoising Diffusion Models for Combinatorial Optimization. Denoising Diffusion Models (DDMs) [9] represent a powerful class of generative models that are primarily known from applications in image and video generation. They are trained by adding noise to labeled training data (forward process) and learning to reverse this process (backward process, i.e., denoising) in a stepwise fashion. A trained model is then iteratively applied to random noise to generate new samples following the probability distribution of the training data. Sun and Yang [24] pioneered the use of DDMs for CO with DIFUSCO, proposing a denoising diffusion model powered by a GNN that can generate high-quality solutions for problems like the TSP and the MISP. Key characteristics include its potential to capture multi-modal solution distributions and faster sampling compared

to autoregressive models. However, DIFUSCO often requires problem-specific post-processing heuristics (e.g., greedy decoding, local search) to ensure feasibility and enhance solution quality. In comparison to autoregressive generative models, denoising diffusion models like DIFUSCO have the advantage of potentially scaling much better to larger instances due to the parallel nature of their solution construction when making use of an efficient GNN implementation on a GPU. Moreover, many diverse solution candidates can be generated in parallel when a powerful-enough GPU is provided.

Subsequent works have sought to improve DIFUSCO. With DISCO, Yu et al. [28] accelerate inference by using residue-guided sampling and analytical denoising steps. The work of [10] employs progressive distillation to compress the diffusion process into fewer steps, achieving substantial speedups while maintaining high solution quality. Others explore alternative learning paradigms; Li et al. [18] combine supervised diffusion learning with an unsupervised gradient-based search during inference, which can improve the quality of the generated solutions. The work of [22] introduces a purely unsupervised diffusion framework that allows training without labeled optimal solutions.

Evolutionary Algorithms. Evolutionary algorithms [1] are metaheuristics inspired by natural evolution. They maintain a population of candidate solutions, iteratively improving them through selection, recombination (crossover), and mutation. Their application to the MISP has been explored using different approaches, such as permutation-based encodings combined with greedy decoders and operators like the partially matched crossover [2], or direct binary representations using penalty functions to handle constraints and standard genetic operators like two-point crossover and bit-flip mutation [20].

ML-EA Hybridization. The integration of machine learning (ML) with EAs has emerged as a powerful paradigm for solving complex optimization problems. Recent surveys, such as [19], systematically analyze surrogate-assisted evolutionary algorithms that employ ML models to approximate expensive fitness evaluations, significantly reducing computational costs. Beyond surrogate assistance, ML techniques have been integrated into EA operators, including adaptive parameter control strategies and reinforcement learning-guided search mechanisms [3]. A major departure from traditional EA paradigms is the Learnable Evolution Model (LEM) [21], which replaces stochastic operators with inductive learning to generate targeted offspring, enabling “quantum leaps” in fitness.

3 The DDEA Framework

This section details DDEA, our proposed framework integrating pre-trained DDMs into the classical EA cycle. The primary motivation is to leverage the powerful generative capabilities of DDMs, which learn rich distributions over complex solution spaces, to enhance the exploration and exploitation abilities of EAs for challenging combinatorial optimization problems.

The core idea of DDEA is not to replace the entire EA but to augment or substitute specific components—namely initialization and recombination—with operations guided by a DDM pre-trained on the target problem distribution. This hybrid approach aims to combine the global search perspective inherent in EAs with the high-quality, structure-aware sampling provided by DDMs.

The DDEA retains a standard EA loop. Parent solutions are selected by binary tournaments, the diffusion-based recombination operator is applied, followed by a standard mutation operator, and we replace with elitism, never accepting duplicate solutions. Figure 1 provides a high-level comparison between the workflow of a standard EA and the DDEA.

The key strength of DDEA lies in its problem independence – the framework can be applied to any combinatorial optimization problem with minimal adaptations. Only two problem-specific components are required: a simple decoding strategy to transform probability vectors into feasible solutions, and a source of training labels (which can be obtained from any specialized solver).

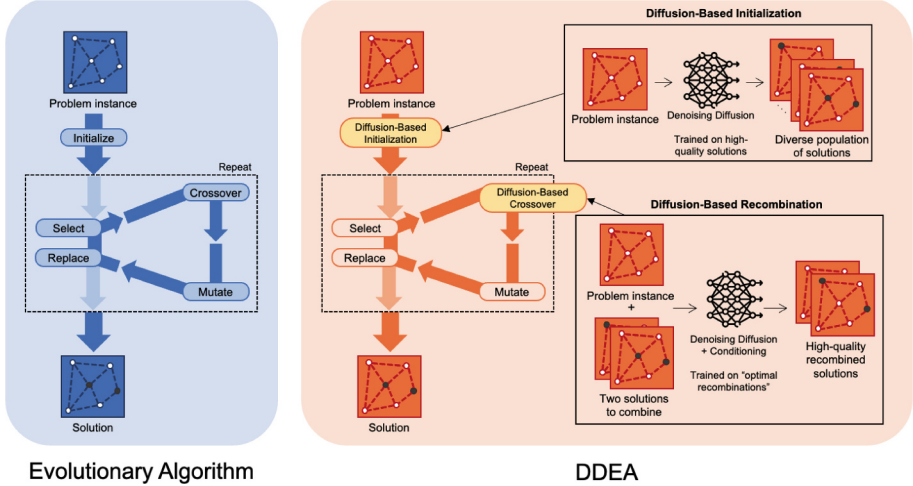


Fig. 1. Overview of the DDEA design in contrast to a classical EA. DDEA replaces standard initialization and recombination with diffusion-based counterparts.

3.1 The Maximum Independent Set Problem, Its Solution Representation, and Solution Decoding

We use the MISPP here as an exemplary problem to illustrate the DDEA framework and to make concrete experiments. This problem is defined on an undirected graph $G = (V, E)$, and its goal is to find the largest subset of vertices $S \subseteq V$ such that no two vertices in S are adjacent. Formally, we want to maximize the size of S while ensuring that for all pairs of vertices $u, v \in S$, $(u, v) \notin E$.

We encode candidate solutions as a binary incidence vector of size $n = |V|$, where each element x_i represents the presence (1) or absence (0) of node $i \in V$ in the independent set.

A problem-specific component of our framework is the decoding process that transforms a vector of node probabilities obtained from a DDM into a feasible solution. This decoding process operates as follows. Given an input probability vector $\mathbf{p} \in [0, 1]^n$, which represents the likelihood of each node to be in the independent set, we first sort all nodes in descending order of their probability scores. Starting with an empty solution, we then process nodes in this order. For each node i , if it has not been marked as ineligible, we include it in our solution and mark all its neighbors as ineligible (since including any neighbor would violate the independence constraint). This greedy construction ensures that high-probability nodes are prioritized while maintaining feasibility at each step. The resulting solution is always a valid independent set, making the population remain feasible at all times. This decoding algorithm can be vectorized over the population, achieving sublinear time growth in the population size P .

The DDEA employs a standard mutation operator, which is applied with a fixed probability p_m to each solution in the population. If a solution x is selected for mutation, each vertex selected in the current solution ($x_i = 1$) is deselected with probability p_d . The mutated solution x' is the result of decoding a probability vector $\mathbf{p}$. This vector $\mathbf{p}$ is sampled from $\mathcal{U}(0, 1)^n$, with the modification that $\mathbf{p}[i]$ is forced to 0 if $x_i = 1$ and vertex i is deselected, and forced to 1 if $x_i = 1$ and vertex i is not deselected.

3.2 Diffusion-Based Initialization

To generate a high-quality and diverse initial population, DDEA leverages a pre-trained DIFUSCO model [24]. Crucially, DIFUSCO exhibits the multi-modal property [24, 28], enabling it to sample diverse solutions from the underlying distribution when starting from different noise. This makes it well-suited for initializing an EA population.

The initialization involves two steps. First, we sample node probability vectors (heatmaps) $\mathbf{p}_j \in [0, 1]^n$, $j = 1, \dots, P$ from the pre-trained DIFUSCO model for the target MISP efficiently in parallel, where P denotes the population size. Second, we decode all these heatmaps $\mathbf{p}_j$ into feasible candidate solutions $x_j \in \{0, 1\}^n$ using the above described decoding heuristic, ensuring all initial individuals are valid independent sets.

3.3 Diffusion-Based Recombination

Recombination (or crossover) in genetic algorithms is the major operator to generate promising offspring by inheriting and combining properties of usually two parental candidate solutions. Traditional crossover operators (like uniform or single-point crossover) are often generic and may not effectively preserve or combine beneficial structural information specific to the problem domain (e.g., graph structures in MISP).

The goal of diffusion-based recombination is to be a more intelligent and problem-aware, learned operator. Instead of simple bit mixing in a naive randomized fashion, we use a conditional diffusion model to generate offspring. This model is trained to sample high-quality solutions from a distribution conditioned on the provided parent solutions, in the following labeled x and y . The intuition is that the model learns how to effectively merge high-quality features of both parents within the learned manifold of high-quality MISP solutions.

Optimized Recombination as an Expert Demonstrator. To train our diffusion recombination model, we require training samples of high-quality offspring corresponding to given parent pairs. We generate this training data using an *expert demonstrator* – an operator designed to produce high-quality offspring, even if computationally expensive. This demonstrator, that we refer to as “optimized recombination”, is realized by means of an Integer Linear Program (ILP). It tries to find the best solution in a restricted search space centered around two parent solutions. Let $x, y \in \{0, 1\}^n$ be the parent solutions for a MISP instance given by a graph $G = (V, E)$, and $z \in \{0, 1\}^n$ be the decision variables for the offspring. Let $S_v = \{i \in V \mid v_i = 1\}$ be the solution set for a given incidence vector v and $U_v = V \setminus S_v$ be the corresponding unselected nodes. The ILP is defined as follows:

$$\max \sum_{i \in V} z_i \quad (1)$$

$$\text{s.t. } z_u + z_v \leq 1 \quad \forall (u, v) \in E \quad (2)$$

$$\underbrace{\left(\sum_{i \in S_x} (1 - z_i) + \sum_{i \in U_x} z_i \right)}_{h(z, x)} + \underbrace{\left(\sum_{i \in S_y} (1 - z_i) + \sum_{i \in U_y} z_i \right)}_{h(z, y)} \leq k \quad (3)$$

$$k = \lambda \cdot \left(\sum_{i \in S_x} (1 - y_i) + \sum_{i \in U_x} y_i \right) \quad (4)$$

$$z_i \in \{0, 1\} \quad \forall i \in V \quad (5)$$

The objective function (1) maximizes the offspring independent set size. Constraints (2) and (5) enforce offspring feasibility. Inspired by local branching [7], constraint (3) limits the combined Hamming distance $h(z, x) + h(z, y)$ between child z and parents x, y to k . The threshold k is defined in (4) as a factor λ of the parent distance $h(x, y)$. Parameter λ controls exploration: larger values allow more deviation from parents while smaller values restrict the search space. Solving this ILP with an exact solver (subject to a time limit) yields a high-quality offspring z^* , which serves as the expert target for training the diffusion model.

The Diffusion Recombination Model Architecture. To enable parent conditioning in the recombination model, we extend DIFUSCO’s input features to include each node’s status in both parent solutions x and y alongside its positional embedding. The three features are independently embedded to dimension d_h

and summed to form the initial node representation, following [25]. This modified input allows the GNN layers to process the graph structure concurrently with parent solution information during denoising steps. The remaining diffusion architecture (GNN layers, time embedding, output layers) and the denoising process follow the practices outlined in [24].

Training via Imitation Learning. In order to train using imitation learning, we first generate a dataset of expert demonstrations. We consider a DDEA variant where we apply the optimized recombination operator, i.e., the ILP from above, as the recombination. During runs of this DDEA variant, the ILP recombination operator is invoked to compute the expert offspring z^* for each selected parent pair (x, y) , and the resulting triplets (x, y, z^*) , along with the corresponding graph G , are stored to form the training dataset. This dataset then forms the basis for training the diffusion model. Its generation is computationally expensive, but we argue that it only needs to be done once offline on representative instances, so this runtime is not a significant concern. More specifically, a GNN model is trained to iteratively predict for two given parents x and y and some initial uniform noise how to reduce noise to ultimately obtain z^* , conditioned on time steps and the graph G besides the parent solutions x and y .

During the recombination phase of the DDEA, given a selected parent pair (x, y) , we use the trained conditional diffusion model to sample one or more offspring heatmaps $\mathbf{p}$. These heatmaps are then converted into feasible offspring by again applying the decoding heuristic. This approach allows DDEA to generate new solutions that are not random combinations of parents but are guided by the learned distribution of high-quality solutions conditioned on those parents.

4 Experimental Evaluation

We evaluate our DDEA framework on the MISP using Erdős-Rényi (ER) random graphs of varying sizes. Experiments focus on evaluating the core diffusion components, performing ablation studies, benchmarking against other methods, and evaluating out-of-distribution generalization.

4.1 Setup

Datasets. We use Erdős-Rényi (ER) graphs generated as in [6] using edge probability 0.15. Three sets of graphs were generated with nodes n sampled from [50, 100], [300, 400], and [700, 800]. For each range, 40000 graphs were created, keeping 128 for testing. For ER-700-800, we reuse the testing dataset of Sun and Yang [24] and only generate the training set. Ground truth labels are obtained using KaMIS [17] with a 60 s time limit. We use an Out-of-Distribution dataset of 42 ER graphs with n from [1300, 1400] for generalization tests.

Implementation. Our DDEA implementation uses PyTorch Geometric and EvoTorch, with DDMs based on DIFUSCO [24]. We used Gurobi¹ 12.0 for the ILP-based optimized recombination operator in single-threaded mode with a 15 s time limit. DDM inference and training used an NVIDIA A100 80 GB GPU. Other experiments used a cluster with Intel(R) Xeon(R) E5-2640 v4 CPU at 2.40 GHz and 160 GB RAM on Ubuntu 18.04.6 LTS.

DDEA Configuration. We defer the discussion on the population size P and the number of generations G to the next section. The mutation probability is $p_m = 0.1$ and the deselection probability $p_d = 0.05$. Moreover, for the allowed deviation from parental solutions in expert recombination training, we set $\lambda = 1.75$. These parameters were identified by preliminary experiments and limited tuning. The DDMs use DIFUSCO’s anisotropic GNN architecture with edge gating, hidden dimension $d_h = 256$ and $n_l = 12$ layers. Following [24], we use Gaussian diffusion for ER graphs. For inference, we use a cosine schedule with $T_{\text{inf}} = 50$ denoising steps. For training, we use a linear schedule with $T_{\text{train}} = 1000$ steps and cosine inference schedule, batch size 64, learning rate 0.0002, and cosine-decay with weight decay 0.0001, as in [24].

Metrics. We measure solution quality using the gap to the best-known solution (BKS): $\% \text{-Gap} = 100\% \cdot \frac{\text{BKS-size} - \text{solution-size}}{\text{BKS-size}}$. For final benchmarking, we follow [24] and report mean cost. We also use the Primal Integral [5], normalized by the area of the rectangle defined by the best-known solution value: $\frac{\sum_{i=1}^n a_{i-1} \cdot (t_i - t_{i-1})}{(t_n - t_0) \cdot \text{BKS-size}}$, where a_i is the incumbent solution value at time t_i .

Baselines for the Recombination and Initialization Operators. For ablation, we use baseline operators to serve as non-diffusion counterparts. For initialization, we use a randomized greedy heuristic (RG) generating probability vectors decoded via the greedy heuristic from Sect. 3.1. For recombination, we use the Consensus-Divergence Crossover (CDX), which takes two parents, identifies common selected vertices, and generates two offspring probability vectors. The first offspring assigns probability one to vertices selected in both parents to strongly encourage their selection (consensus), while the second offspring assigns probability zero to these common vertices to encourage diversity (divergence). We finally decode the offspring using the greedy decoding.

4.2 Evaluation of the Diffusion-Based Recombination

We first evaluate the diffusion recombination operator in isolation. It is trained on ≈ 160000 examples from expert ILP recombination across all three datasets. Table 1 compares the gaps to the expert recombination label against DIFUSCO x2 inference and CDX baseline using parents of varying quality: high-quality EA parents, medium-quality heuristic solutions from decoded random vectors,

¹ <https://www.gurobi.com>.

and low-quality random solutions. The results show that the diffusion recombination effectively approximates the expert operator, outperforming CDX and DIFUSCO. Performance degrades with decreasing parent quality, demonstrating cost monotonicity and validating the design of the operator.

Table 1. Average gap (%) to the optimized recombination label for different approaches and parent types.

Method	Parent Type	Dataset / Gap (%)		
		ER-50-100	ER-300-400	ER-700-800
Diffusion Recombination	EA Parents	1.28 ± 3.34	6.10 ± 8.53	5.18 ± 8.48
	Heuristic	2.86 ± 4.40	12.18 ± 6.86	16.81 ± 6.38
	Random	3.10 ± 5.09	26.28 ± 5.90	26.60 ± 4.15
DIFUSCO	–	6.36 ± 5.54	8.51 ± 7.17	5.44 ± 4.89
CDX Recombination	EA Parents	6.38 ± 7.25	20.52 ± 7.61	24.13 ± 6.03

4.3 Evaluating DDEA Components Within the EA Framework

Table 2 assesses the contribution of the diffusion-based initialization (DI) and recombination (DR) within the complete EA framework. We compare DDEA ($P = 16$, $G = 20$) against the following variants:

- DDEA without DR: Uses DI with CDX recombination (DI / CDX)
- DDEA without DI: Uses DR with RG initialization (RG / Diffusion)
- Naïve EA: Uses RG initialization and CDX recombination (RG / CDX)
- Expert Recombination EA: Uses DI and ILP recombination (DI / Optim.)

The table shows gaps, hyperparameters, total runtime t and runtime per generation t/gen . The results show that full DDEA achieves performance comparable to expert optimized recombination, with minimal gap differences. The learned diffusion recombination effectively approximates optimized recombination while drastically reducing runtime per generation (e.g., 5 s vs. 88 s on ER-300-400). The full DDEA significantly outperforms the Naïve EA variants (RG/CDX and DI/CDX) under matched time limits. Comparing the full DDEA to the DI/CDX variant highlights the substantial benefit of diffusion recombination over classical methods (e.g., 2.81% vs. 5.75% gap on ER-700-800). Similarly, comparing the full DDEA to the RG/DR variant shows the importance of DI (e.g., 0.94% vs. 1.39% gap on ER-300-400). Although diffusion-based initialization is beneficial, the diffusion recombination operator demonstrates robustness, performing well even with greedy initialization (RG/DR) compared to the Naïve baseline (RG/CDX). While the runtime of DDEA is dominated by the diffusion steps during recombination, the learned operator successfully approximates the expensive ILP solving, enabling near-expert performance in more practical runtimes.

Table 2. Performance comparison of different EA variants across datasets.

Dataset	Init.	Recomb.	G	P	%-Gap	t [s]	t/gen [s]
ER-50-100	DI	CDX	50	16	0.55 ± 1.52	24.1 ± 7.5	0.5 ± 0.2
	RG		50	50	0.25 ± 1.15	34.6 ± 5.1	0.7 ± 0.1
	DI	Optim.	20	16	0.00 ± 0.00	11.4 ± 4.0	0.6 ± 0.2
	DI	Diffusion	20	16	0.11 ± 0.53	14.4 ± 1.8	0.7 ± 0.1
	RG		20	16	0.11 ± 0.69	12.5 ± 1.9	0.6 ± 0.1
ER-300-400	DI	CDX	75	16	3.57 ± 2.27	76.6 ± 10.6	1.0 ± 0.1
	RG		75	16	8.58 ± 2.98	48.5 ± 4.5	0.6 ± 0.1
	DI	Optim.	5	16	0.58 ± 1.25	441.4 ± 64.6	88.3 ± 12.9
	DI	Diffusion	20	16	0.90 ± 1.37	188.0 ± 16.9	9.4 ± 0.8
	RG		20	16	1.39 ± 2.09	109.0 ± 27.3	5.4 ± 1.4
ER-700-800	DI	CDX	305	16	5.75 ± 2.66	231.7 ± 33.5	0.8 ± 0.1
	RG		305	16	13.36 ± 3.48	238.0 ± 57.0	0.8 ± 0.2
	DI	Optim.	3	16	3.21 ± 2.09	314.4 ± 27.3	104.8 ± 9.1
	DI	Diffusion	20	16	2.95 ± 2.02	478.2 ± 9.5	23.9 ± 0.5
	RG		20	16	4.52 ± 2.33	473.7 ± 37.1	23.7 ± 1.9

4.4 Comparing to Other Approaches for the MISP

We start by comparing DDEA against DIFUSCO with different population sizes $P \in \{16, 24, 32, 48\}$. Though these are small values compared to standard EAs, they allow DDEA to operate in reasonable runtimes, and they are sufficient to achieve excellent results. We compare against DIFUSCO sampling x32 solutions in parallel, running it until matching the runtime of DDEA. Figure 2 shows the comparison of average independent set (IS) size against different runtime budgets for DDEA and DIFUSCO. For DDEA, each plotted datapoint is the average of the cost and the average of the runtime at each generation. For DIFUSCO, each datapoint is the average of the cost and the average runtime at the end of a diffusion sampling call. A consistent pattern across datasets shows that DDEA achieves superior solution quality in the early generations. However, with certain suboptimal parameter settings, DIFUSCO’s multi-sampling approach can eventually outperform DDEA after extended runtime. On ER-50-100, two generations are enough for DDEA ($P = 24$) to achieve an optimal solution. In this case, DDEA outperforms DIFUSCO on a time budget of 13s. On ER-300-400, both DDEA ($P = 16$, $P = 24$) are overtaken by DIFUSCO after a number of generations. However, DDEA ($P = 32$, $P = 48$) are able to outperform DIFUSCO on all the considered time budgets. Interestingly, $P = 32$ seems to be the optimal population size for this dataset, as increasing it to $P = 48$ does not improve the results. Finally, on ER-700-800, DDEA ($P = 32$, $P = 48$) are again able to outperform DIFUSCO on all the considered time budgets, but this time, larger population sizes do improve the costs. On the question of whether DIFUSCO

could outperform DDEA with even longer time budgets, we argue that, even if theoretically possible, it would require impractical computational time in the current experimental setup.

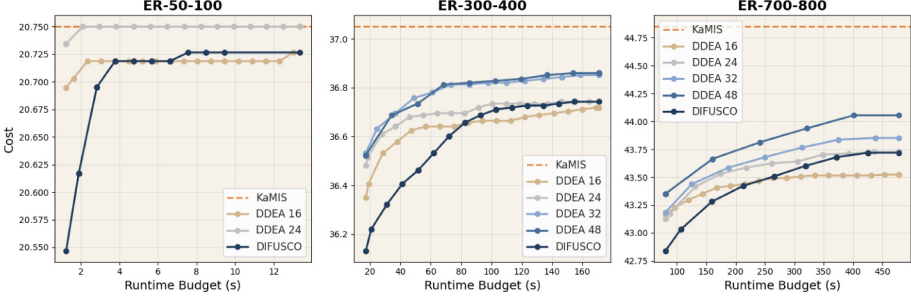


Fig. 2. Average cost (IS size, higher is better) against different runtime budgets for DDEA and DIFUSCO.

Related to Fig. 2, Table 3 shows the primal integral for the different methods, which is a runtime-agnostic metric. As mentioned above, the optimal configurations seem to be $P = 16, 32, 48$ for ER-50-100, ER-300-400, and ER-700-800 respectively. These configurations significantly outperform DIFUSCO.

Table 3. Comparison of primal integral (%) across datasets.

Method	Dataset / Primal Integral (%)		
	ER-50-100	ER-300-400	ER-700-800
DDEA ($P = 16$)	0.15	1.10	3.12
DDEA ($P = 24$)	0.00	0.94	2.81
DDEA ($P = 32$)	—	0.70	2.62
DDEA ($P = 48$)	—	0.70	2.27
DIFUSCO	0.19	1.21	3.07

After showing the dominance of DDEA over DIFUSCO, independently of the time budget, we now consider a version of DDEA as follows: ($P = 24$, $G = 15$) for ER-50-100, ($P = 32$, $G = 15$) for ER-300-400, and ($P = 20$, $G = 10$) for ER-700-800, and we will denote it as “DDEA-long”, since it uses relaxed runtime budgets. We compare against “DIFUSCO-timed”, which samples 32 solutions in parallel as long as the DDEA-long runs. Additionally, we benchmark DIFUSCO x16, Gurobi solving a standard MISP ILP model (timed to the same runtime as DDEA-long), and KaMIS (timed to 60s).

Table 4 shows the average cost, the runtime (T), and the number of evaluations (E) and diffusion calls (D) for the different methods. Note that the number

of diffusion calls (D) should be interpreted with caution, as different population sizes (P) result in different batch sizes during sampling, which affects the actual runtime of each diffusion call. We again observe that DDEA-long outperforms DIFUSCO x16 and DIFUSCO-timed. With respect to the latter one, DDEA provides IS sizes that are 0.25% and 0.31% larger on ER-300-400 and ER-700-800 respectively, with slightly shorter runtimes and fewer individuals explored. DDEA-long also outperforms Gurobi in the ER-300-400 dataset and in the ER-700-800 dataset. DDEA’s IS sizes are 3.90% and 7.50% larger respectively. Moreover, it matches the costs of both Gurobi and KaMIS on ER-50-100, solving the dataset to optimality.

Table 4. Comparison of average independent set size (column Cost, higher is better), runtime (column T in seconds), number of function evaluations (column E), and number of diffusion calls (column D) for different MISP solvers.

Method	ER-50-100				ER-300-400				ER-700-800			
	Cost	T	E	D	Cost	T	E	D	Cost	T	E	D
DIFUSCO x16	20.32	0.7	16	1	35.53	8.7	16	1	42.12	43.6	16	1
DIFUSCO-timed	20.73	13.9	640	20	36.76	207.1	960	30	43.99	851.3	960	30
DDEA-long	20.75	13.3	540	16	36.85	183.3	720	16	44.13	803.1	720	11
KaMIS (60 s)	20.75	60.0	—	—	37.05	60.0	—	—	44.85	60.0	—	—
Gurobi	20.75	0.1	—	—	35.47	171.2	—	—	41.06	803.6	—	—

4.5 Out-Of-Distribution Generalization

Finally, we test DDEA’s generalization on the ER-1300-1400 dataset, which is larger than the training instances (with a maximum of 800 nodes). Both DDEA and DIFUSCO use diffusion models trained on ER-700-800. Table 5 compares DIFUSCO (x16 and timed mode) against DDEA ($P = 16$, $G = 10$) on ER-1300-1400, showing average cost, runtime (T), evaluations (E), and diffusion calls (D). DIFUSCO-timed gains only +1.62 with respect to DIFUSCO x16 despite 839 extra seconds runtime. With similar runtime, DDEA achieves notably higher costs. The primal integrals (PIs) confirm this: DDEA’s PI (2.27) exceeds DIFUSCO’s (1.21) with fewer evaluations and diffusion calls. This shows DDEA effectively uses evolutionary search to find solutions beyond the training distribution.

Table 5. Comparison of average independent set size (column Cost, higher is better), runtime (column T in seconds), number of function evaluations (column E), and number of diffusion calls (column D) on the out-of-distribution dataset ER-1300-1400.

Method	Cost	T	E	D
DIFUSCO $\times 16$	37.07	108.1	16	1
DIFUSCO-timed	38.69	947.1	384	12
DDEA (P = 16, G = 10)	43.17	909.5	240	21
KaMIS (60 s)	49.91	60.0	–	–

5 Conclusion

We introduced DDEA, a framework integrating pre-trained denoising diffusion models into evolutionary algorithms for combinatorial optimization. DDEA leverages diffusion models in two ways: (1) Diffusion-based initialization uses a pre-trained DDM to sample a diverse, high-quality initial population. (2) Diffusion-based recombination employs a novel operator, trained via imitation learning using an ILP-based expert demonstrator, to recombine parent solutions into promising offspring.

Experimental results on the MISP demonstrate DDEA’s efficacy. When given the same runtime budget, DDEA outperforms both DIFUSCO and Gurobi. Moreover, DDEA exhibits better out-of-distribution generalization compared to DIFUSCO, showcasing robustness gained from evolutionary search. Ablation studies confirm that both diffusion initialization and recombination components are crucial, with recombination being particularly impactful.

Denoising diffusion-based approaches may scale better to larger problems than autoregressive ML techniques, as the latter are inherently sequential. Despite our promising results, our DDEA implementation faces limitations. The main bottleneck is computational cost; the diffusion inference steps, especially for recombination, dominate the runtime and require GPU resources, hindering scalability. A reason for this is that the current implementation suffers from inefficient in-GPU parallelization. Better GPU utilization could yield significant improvements. Additionally, generating training data for recombination via imitation learning is expensive, requiring EA runs with costly ILP solver calls.

This opens many opportunities for future work. We can optimize diffusion inference parallelization for substantial runtime improvements. Exploring cheaper methods for generating recombination training data, perhaps via a baseline metaheuristic, is crucial for practicality. Applying DDEA to other CO problems appears highly promising, as the framework is generic. Investigating diffusion-based mutation operators and integrating newer, more powerful diffusion backbones (e.g., T2T [18], DISCO [28]) could further enhance performance.

In conclusion, DDEA highlights the potential of combining diffusion models’ generative strengths with EAs’ robust search capabilities. This hybrid ML-metaheuristic approach offers a promising direction for developing effective CO

solvers, bridging the gap between pure learning-based methods and specialized classical algorithms.

Data and code availability The code and problem instances used in this work are available at https://github.com/jsalvasoler/difusco_ddea.

References

1. Back, T., Fogel, D., Michalewicz, Z.: Handbook of Evolutionary Computation. Oxford University Press, New York, NY (1997)
2. Bäck, T., Khuri, S.: An evolutionary heuristic for the maximum independent set problem. In: Proceedings of the First IEEE Conferences on Evolutionary Computation. IEEE World Congress on Computational Intelligence, vol.2 , pp. 531–535 (1994)
3. Bai, H., Cheng, R., Jin, Y.: Evolutionary reinforcement learning: a survey. *Intelligent Computing* **2**, 0025 (2023)
4. Bengio, Y., Lodi, A., Prouvost, A.: Machine learning for combinatorial optimization: a methodological tour d’horizon. *Eur. J. Oper. Res.* **290**(2), 405–421 (2021)
5. Berthold, T., Csizmadia, Z.: The confined primal integral: a measure to benchmark heuristic MINLP solvers against global MINLP solvers. *Math. Program.* **188**(2), 523–537 (2021). <https://doi.org/10.1007/s10107-020-01547-5>
6. Böther, M., Kißig, O., Taraz, M., Cohen, S., Seidel, K., Friedrich, T.: What’s wrong with deep learning in tree search for combinatorial optimization. In: Proceedings of the International Conference on Learning Representations (ICLR) (2022)
7. Fischetti, M., Lodi, A.: Local branching. *Math. Program.* **98**(1), 23–47 (2003)
8. Georgiev, D., Numeroso, D., Bacciu, D., Lió, P.: Neural algorithmic reasoning for combinatorial optimisation. In: Proceedings of the Second Learning on Graphs Conference (LoG 2023) vol. 231, PMLR (2023)
9. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. In: Advances in Neural Information Processing Systems, pp. 6840–6851 (2020)
10. Huang, J., Sun, Z., Yang, Y.: Accelerating diffusion-based combinatorial optimization solvers by progressive distillation. In: Proceedings of the ICML 2023 Workshop: Sampling and Optimization in Discrete Space (2023)
11. Jin, Y., Yan, X., Liu, S., Wang, X.: A unified framework for combinatorial optimization based on graph neural networks. arXiv preprint [arXiv:2406.13125](https://arxiv.org/abs/2406.13125) (2024)
12. Karalias, N., Loukas, A.: Erdős goes neural: an unsupervised learning framework for combinatorial optimization on graphs. In: Advances in Neural Information Processing Systems (2020)
13. Kemminer, R., Lange, J., Kempkes, J., et al.: Configuring mixed-integer programming solvers for large-scale instances. *Oper. Res. Forum* **5**, 48 (2024)
14. Khalil, E., Dai, H., Zhang, Y., Dilkina, B., Song, L.: Learning combinatorial optimization algorithms over graphs. In: Advances in Neural Information Processing Systems (2017)
15. Khalil, E.B., Bodic, P.L., Song, L., Nemhauser, G., Dilkina, B.: Learning to branch in mixed integer programming. In: Proceedings of the 30th AAAI Conference on Artificial Intelligence, pp. 724–731. AAAI Press (2016)
16. Kool, W., van Hoof, H., Welling, M.: Attention, learn to solve routing problems! In: Proceedings of International Conferences on Learning Representations (ICLR) (2019)

17. Lamm, S., Sanders, P., Schulz, C., Strash, D., Werneck, R.F.: Finding near-optimal independent sets at scale. *J. Heuristics* **23**(4), 207–229 (2017)
18. Li, Y., Guo, J., Wang, R., Yan, J.: From distribution learning in training to gradient search in testing for combinatorial optimization. In: *Advances in Neural Information Processing Systems* (2023)
19. Liang, J., Lou, Y., Yu, M., Bi, Y., Yu, K.: A survey of surrogate-assisted evolutionary algorithms for expensive optimization. *J. Membr. Comput.* (2024). <https://doi.org/10.1007/s41965-024-00155-x>
20. Liu, X., Sakamoto, A., Shimamoto, T.: A genetic algorithm for maximum independent set problems. *Inf. Intell. Syst.* **3**, 1916–1921 (1996)
21. Michalski, R.S.: Learnable evolution model: evolutionary processes guided by machine learning. *Mach. Learn.* **38**, 9–40 (2000)
22. Sanokowski, S., Hochreiter, S., Lehner, S.: A diffusion model framework for unsupervised neural combinatorial optimization. In: *Proceedings of International Conferences on Machine Learning (ICML)*, pp. 43346–43367 (2024)
23. Scavuzzo, L., Aardal, K., Lodi, A., Yorke-Smith, N.: Machine learning augmented branch and bound for mixed integer linear programming. *Math. Program.* (2024). <https://doi.org/10.1007/s10107-024-02130-y>
24. Sun, Z., Yang, Y.: DIFUSCO: Graph-based diffusion solvers for combinatorial optimization. In: *Advances in Neural Information Processing Systems*, pp. 3706–3731 (2023)
25. Vaswani, A., et al.: Attention is all you need. In: *Advances in Neural Information Processing Systems* (2017)
26. Vinyals, O., Fortunato, M., Jaitly, N.: Pointer networks. In: *Advances in Neural Information Processing Systems*, pp. 2692–2700. Curran Associates, Inc. (2015)
27. Ye, H., et al.: Reevo: large language models as hyper-heuristics with reflective evolution. In: *Advances in Neural Information Processing Systems* (2024)
28. Yu, K., Zhao, H., Huang, Y., Yi, R., Xu, K., Zhu, C.: Disco: efficient diffusion solver for large-scale combinatorial optimization problems. *arXiv preprint* (2024). <https://doi.org/10.48550/arXiv.2406.19705>
29. Zhang, D., Dai, H., Malkin, N., Courville, A., Bengio, Y., Pan, L.: Let the flows tell: solving graph combinatorial optimization problems with GFlowNets. In: *Advances in Neural Information Processing Systems*, pp. 11952–11969 (2023)